

I'll build you a comprehensive mental model for how objects actually work in real JavaScript programs. This is about *thinking like a developer*, not memorizing syntax.

The Core Mental Model: Objects as "Labeled Containers of State"

Forget the beginner definition ("objects are key-value pairs"). Here's how experienced developers actually think about them:

An object is a named bundle of related state that travels together through your program.

When you see `user`, you don't think "a collection of properties." You think: *"This is the current user entity. It has an identity, and it carries all its relevant data with it."*

Why Objects Exist (The Real Reason)

Variables are isolated. In real programs, data is relational.

```
// Without objects: chaos
let userName = "Alice";
let userEmail = "alice@example.com";
let userAge = 28;
let userIsActive = true;

// Function needs 4 parameters, order matters, easy to mix up
function sendEmail(name, email, age, isActive) { ... }

// With objects: cohesion
const user = {
  name: "Alice",
  email: "alice@example.com",
  age: 28,
  isActive: true
};

// One parameter, one concept, impossible to mix up
function sendEmail(user) { ... }
```

The insight: Objects exist because *real-world entities have multiple attributes that change together and travel together*. A user isn't a name, or an email, or an age. A user is an entity that *has* all of those things.

When to Use Objects vs. Variables

Use variables when...	Use objects when...
The data is truly independent	Multiple values describe one entity
The value is a simple primitive	The data needs to be passed around as a unit

Use variables when...	Use objects when...
No relationship between values	Values change together logically
<code>const pi = 3.14</code>	<code>const config = { apiUrl, timeout, retries }</code>

Common mistake: Creating objects for single values, or using separate variables for clearly related data.

```
// BAD: These are clearly related, but scattered
let productName = "Laptop";
let productPrice = 999;
let productStock = 12;

// GOOD: One entity, one container
const product = {
  name: "Laptop",
  price: 999,
  stock: 12
};

// ALSO BAD: Over-engineering a single value
const singleValue = { value: 42 }; // Just use const value = 42;
```

How Properties Actually Work (The Hidden Mechanics)

Adding Properties: Objects Are Expandable Bags

```
const user = { name: "Alice" };

// Adding properties dynamically
user.email = "alice@example.com";
user["age"] = 28;

// Real-world pattern: Building objects incrementally
function createUserFromForm(formData) {
  const user = {};

  // Properties added conditionally based on what exists
  if (formData.name) user.name = formData.name;
  if (formData.email) user.email = formData.email;
  if (formData.preferences) user.preferences = formData.preferences;

  // Default properties added automatically
  user.createdAt = new Date();
  user.isActive = true;
  user.id = generateId();
}
```

```
    return user;
}
```

The mental model: Objects aren't fixed schemas like database tables. They're expandable bags. You add what you need, when you need it. JavaScript doesn't complain about missing properties—it just returns `undefined`.

Dynamic Access: The Bracket Notation Superpower

```
const user = { name: "Alice", email: "alice@example.com", age: 28 };

// Dot notation: static, known keys
user.name;

// Bracket notation: dynamic, computed keys
const field = "email";
user[field];          // "alice@example.com"

// Real-world use: Config-driven property access
const config = {
  sortBy: "age",
  order: "desc"
};

function sortUsers(users, config) {
  return users.sort((a, b) => {
    const field = config.sortBy; // "age"
    const aVal = a[field];      // dynamic access: user.age
    const bVal = b[field];
    return config.order === "desc" ? bVal - aVal : aVal - bVal;
  });
}
```

Critical insight: Bracket notation is how you write *generic* code that works with any property. It's the difference between hardcoding and being flexible.

Reference vs. Value: The Most Important Concept

This is where most developers get confused. Understand this deeply.

Primitives (Value Behavior)

```
let a = 5;
let b = a;    // b gets a COPY of the value 5
b = 10;
console.log(a); // 5 (unaffected)
```

Objects (Reference Behavior)

```
const user1 = { name: "Alice" };
const user2 = user1;    // user2 gets a REFERENCE to the same object

user2.name = "Bob";
console.log(user1.name); // "Bob" (same object!)

// Both variables point to the SAME object in memory
console.log(user1 === user2); // true
```

The mental model: Variables don't "contain" objects. They *point to* objects in memory. When you assign `user2 = user1`, you're not copying the object. You're copying the *pointer* (the arrow pointing to the object).

Memory visualization:

user1 → { name: "Alice" } ← user2

Why This Matters in Real Programs

```
// Scenario: Updating a user in a list
const users = [
  { id: 1, name: "Alice" },
  { id: 2, name: "Bob" }
];

const userToUpdate = users[0]; // userToUpdate points to the SAME object
userToUpdate.name = "Alicia";

console.log(users[0].name); // "Alicia" – the array was modified!
```

This is both powerful and dangerous. You can modify objects "remotely" through any reference. But you can also accidentally mutate data you didn't mean to touch.

Object Mutation: The Double-Edged Sword

The Problem: Silent Side Effects

```
function addTimestamp(user) {
  user.lastLogin = new Date(); // MUTATES the original object
  return user;
}

const alice = { name: "Alice" };
```

```
addTimestamp(alice);

// alice is now modified – maybe unexpectedly
console.log(alice.lastLogin); // Date object
```

The mistake: Functions that *appear* to just return data but secretly modify their inputs. This causes bugs that are hard to trace because the mutation happens far from where the problem manifests.

The Solution: Defensive Patterns

```
// Pattern 1: Explicit mutation (when intended)
function updateUser(user, updates) {
  Object.assign(user, updates); // Mutates user, but caller knows
  return user;
}

// Pattern 2: Return new object (no mutation)
function withTimestamp(user) {
  return {
    ...user, // Shallow copy
    lastLogin: new Date()
  };
}

const alice = { name: "Alice" };
const updatedAlice = withTimestamp(alice);

// alice is untouched
console.log(alice.lastLogin); // undefined
console.log(updatedAlice.lastLogin); // Date object
```

When to mutate vs. copy:

- **Mutate** when the object is clearly "owned" by the current scope (e.g., local accumulator, internal state)
- **Copy** when the object came from elsewhere (function parameters, external data)

Objects and Functions: The Four Fundamental Patterns

Pattern 1: Functions Receiving Objects

```
// BAD: Destructuring in parameters hides the object nature
function process(name, email, age) { ... }

// GOOD: Receive the object, use it as a unit
function processUser(user) {
  // Now you have the WHOLE user, can access any property
  // Can pass user to other functions
  // Can add properties to user
```

```
// Can check if user has optional properties
}

// Even better: Destructure inside when you need specific parts
function processUser(user) {
  const { name, email } = user;
  // But you still have access to `user` if needed
}
```

Why: When you receive an object, you preserve the *entity*. You can pass it to `sendEmail(user)`, `logActivity(user)`, `checkPermissions(user)` without breaking it apart.

Pattern 2: Functions Returning Objects

```
// Factory pattern: Creating configured objects
function createProduct(name, price, category) {
  return {
    id: generateId(),
    name,
    price,
    category,
    createdAt: new Date(),
    inStock: true,
    // Computed property based on inputs
    displayPrice: `$$${price.toFixed(2)}$`
  };
}

// Builder pattern: Incremental object construction
function createOrder(customerId) {
  const order = {
    id: generateId(),
    customerId,
    items: [],
    status: "pending",
    createdAt: new Date()
  };

  // Return methods that close over the object
  return {
    addItem(product, quantity) {
      order.items.push({ product, quantity });
      return this; // Chainable
    },
    setShipping(address) {
      order.shippingAddress = address;
      return this;
    },
    build() {
      return order;
    }
  }
}
```

```
};  
}  
  
// Usage  
const order = createOrder("cust_123")  
  .addItem({ name: "Laptop", price: 999 }, 1)  
  .addItem({ name: "Mouse", price: 25 }, 2)  
  .setShipping({ city: "NYC", zip: "10001" })  
  .build();
```

Pattern 3: Functions Updating Objects

```
// The "update in place" pattern (mutating)  
function activateUser(user) {  
  user.isActive = true;  
  user.activatedAt = new Date();  
  return user; // Return for chaining  
}  
  
// The "immutable update" pattern (preferred for React, Redux, etc.)  
function activateUserImmutable(user) {  
  return {  
    ...user,  
    isActive: true,  
    activatedAt: new Date()  
  };  
}  
  
// The "batch update" pattern  
function updateUser(user, changes) {  
  return { ...user, ...changes };  
}  
  
// Usage  
const updated = updateUser(alice, {  
  email: "new@example.com",  
  age: 29  
});
```

Pattern 4: Functions Creating Objects (Factories vs Classes)

```
// Factory function: Pure function, no `new`, no `this`  
function createUser(name, email) {  
  return {  
    name,  
    email,  
    login() {  
      console.log(`${this.name} logged in`);  
    }  
  }  
}
```

```
};  
}  
  
// Constructor function: Old school, rarely used now  
function User(name, email) {  
  this.name = name;  
  this.email = email;  
}  
  
// Class: Modern, but fundamentally the same  
class User {  
  constructor(name, email) {  
    this.name = name;  
    this.email = email;  
  }  
  
  login() {  
    console.log(`${this.name} logged in`);  
  }  
}
```

Key insight: Classes are just syntactic sugar over constructor functions. The mental model is identical: a template that produces objects with shared shape and methods.

Nested Objects: Thinking in Layers

The Mental Model: Objects as Trees

```
const company = {  
  name: "TechCorp",  
  address: {  
    street: "123 Main St",  
    city: "San Francisco",  
    coordinates: {  
      lat: 37.7749,  
      lng: -122.4194  
    }  
  },  
  departments: {  
    engineering: {  
      head: { name: "Alice", role: "CTO" },  
      budget: 500000,  
      teams: ["Frontend", "Backend", "DevOps"]  
    },  
    sales: {  
      head: { name: "Bob", role: "VP Sales" },  
      budget: 300000,  
      teams: ["Enterprise", "SMB"]  
    }  
  }  
}
```

```
}  
};
```

How to read this: `company` is the root. Every nested object is a *sub-entity* with its own identity. `company.address` is an address entity. `company.departments.engineering` is a department entity.

Access pattern: Navigate from root to leaf using dots.

```
// Accessing deeply nested data  
const engHead = company.departments.engineering.head.name;  
  
// Safe navigation (avoiding "cannot read property of undefined")  
const lat = company.address?.coordinates?.lat;  
  
// Dynamic deep access  
const dept = "engineering";  
const team = company.departments[dept].teams[0];
```

Common Mistake: Shallow vs Deep

```
const user = {  
  name: "Alice",  
  address: { city: "NYC", zip: "10001" }  
};  
  
// Shallow copy: Only copies the top level  
const copy = { ...user };  
  
copy.name = "Bob";           // Safe: only copy changes  
copy.address.city = "LA";    // DANGER: modifies original too!  
  
console.log(user.address.city); // "LA" – original mutated!  
  
// Why? Because copy.address still points to the SAME nested object  
// user.address → { city: "LA" } ← copy.address
```

The rule: `...spread` only copies one level deep. Nested objects are still shared references.

Arrays of Objects: The Most Common Structure

This is where real programs spend 80% of their time. Master this.

Structure: Array as Collection, Objects as Items

```
// The universal pattern: An array of entities  
const users = [
```

```
{ id: 1, name: "Alice", role: "admin", isActive: true },
{ id: 2, name: "Bob", role: "user", isActive: false },
{ id: 3, name: "Carol", role: "user", isActive: true }
];

// Mental model:
// users    → container (array)
// users[0] → entity (object)
// users[0].name → attribute of that entity
```

Searching: Finding the Right Object

```
// Find ONE object by condition
const alice = users.find(user => user.id === 1);
const firstActive = users.find(user => user.isActive);

// Find INDEX (for updating/removing)
const aliceIndex = users.findIndex(user => user.id === 1);

// Check if ANY match exists
const hasAdmins = users.some(user => user.role === "admin");

// Check if ALL match
const allActive = users.every(user => user.isActive);
```

Filtering: Creating Subsets

```
// New array with only active users
const activeUsers = users.filter(user => user.isActive);

// Chaining: active admins
const activeAdmins = users
  .filter(user => user.isActive)
  .filter(user => user.role === "admin");

// Real-world: Filter then map (common pattern)
const activeUserNames = users
  .filter(user => user.isActive)
  .map(user => user.name); // ["Alice", "Carol"]
```

Updating Objects Inside Arrays

This is the #1 source of confusion. You have an array. You want to change one object. How?

```
// WRONG: Mutates original array directly
users[0].name = "Alicia"; // Silent mutation
```

```
// CORRECT: Immutable update (returns new array)
function updateUserInArray(users, userId, updates) {
  return users.map(user => {
    if (user.id === userId) {
      // Return new object with updates merged
      return { ...user, ...updates };
    }
    return user; // Return unchanged reference
  });
}

const updatedUsers = updateUserInArray(users, 1, { name: "Alicia" });
// users is unchanged, updatedUsers is new array
```

The mental model: Arrays of objects are like a filing cabinet. You don't rewrite the whole cabinet. You pull out one file, modify it, and put it back. But in immutable code, you make a *copy* of the file, modify the copy, and replace the old file with the new one in a new cabinet.

Removing Objects from Arrays

```
// Remove by ID (immutable)
function removeUser(users, userId) {
  return users.filter(user => user.id !== userId);
}

// Remove by index (if you know it)
const withoutFirst = users.slice(1); // New array, skips first

// Remove and return the removed item (mutable, for internal use)
const index = users.findIndex(u => u.id === 2);
const removed = users.splice(index, 1); // removed = [{...Bob}]
```

Finding Specific Objects: The Reference Trap

```
const alice = { id: 1, name: "Alice" };
const users = [alice, { id: 2, name: "Bob" }];

// This works because alice is the SAME object reference
const found = users.find(u => u === alice); // Found!

// But this is more common and safer
const foundById = users.find(u => u.id === 1);
```

Objects Containing Arrays: The Inventory Pattern

```

const shoppingCart = {
  userId: "user_123",
  items: [
    { productId: "p1", name: "Laptop", price: 999, quantity: 1 },
    { productId: "p2", name: "Mouse", price: 25, quantity: 2 }
  ],
  coupon: null,
  status: "active",

  // Methods that operate on the internal array
  get total() {
    return this.items.reduce((sum, item) =>
      sum + (item.price * item.quantity), 0
    );
  },

  addItem(product, quantity = 1) {
    // Check if already in cart
    const existing = this.items.find(item => item.productId === product.id);

    if (existing) {
      existing.quantity += quantity; // Update existing
    } else {
      this.items.push({
        productId: product.id,
        name: product.name,
        price: product.price,
        quantity
      });
    }
  }
};

// Usage
cart.addItem({ id: "p3", name: "Keyboard", price: 75 });
console.log(cart.total); // Computed property

```

The pattern: Object = entity with metadata. Array inside = collection of sub-entities. Methods = operations that maintain consistency between the object and its internal array.

Classes: Why They Create Objects

The Mental Model: Class as Factory Blueprint

```

class Student {
  constructor(name, id) {
    this.name = name;
    this.id = id;
    this.courses = []; // Each instance gets its OWN array
  }
}

```

```

    this.grades = {};
  }

  enroll(course) {
    this.courses.push(course);
  }

  setGrade(course, grade) {
    this.grades[course] = grade;
  }

  getGPA() {
    const grades = Object.values(this.grades);
    if (grades.length === 0) return 0;
    return grades.reduce((a, b) => a + b, 0) / grades.length;
  }
}

// Creating instances: Each is a distinct object
const alice = new Student("Alice", "S001");
const bob = new Student("Bob", "S002");

alice.enroll("Math"); // Only alice's courses changes
bob.enroll("Physics"); // Bob is unaffected

// Storing in arrays: The common pattern
const roster = [alice, bob];

// Finding and operating
const student = roster.find(s => s.id === "S001");
student.setGrade("Math", 95);

```

Why classes? They bundle:

1. **Data structure** (what properties exist)
2. **Initial state** (constructor defaults)
3. **Behavior** (methods that operate on that specific object's data)
4. **Encapsulation** (methods keep the object's data consistent)

Class Instances in Arrays: The Real Power

```

class Library {
  constructor() {
    this.books = []; // Array of objects
    this.members = []; // Array of objects
    this.loans = []; // Array connecting books to members
  }

  addBook(title, author, isbn) {
    const book = { title, author, isbn, isAvailable: true };
    this.books.push(book);
  }
}

```

```
    return book;
  }

  registerMember(name) {
    const member = {
      id: generateId(),
      name,
      borrowedBooks: [],
      joinDate: new Date()
    };
    this.members.push(member);
    return member;
  }

  checkoutBook(memberId, isbn) {
    const member = this.members.find(m => m.id === memberId);
    const book = this.books.find(b => b.isbn === isbn);

    if (!book || !book.isAvailable) {
      throw new Error("Book not available");
    }

    // Update book state
    book.isAvailable = false;

    // Update member state
    member.borrowedBooks.push({
      isbn,
      title: book.title,
      checkoutDate: new Date(),
      dueDate: new Date(Date.now() + 14 * 24 * 60 * 60 * 1000)
    });

    // Record the loan
    this.loans.push({
      memberId,
      isbn,
      checkoutDate: new Date()
    });
  }

  getOverdueLoans() {
    const now = new Date();
    return this.loans.filter(loan => {
      const member = this.members.find(m => m.id === loan.memberId);
      const borrowed = member.borrowedBooks.find(b => b.isbn === loan.isbn);
      return borrowed && borrowed.dueDate < now;
    });
  }
}
```

The insight: The library doesn't just store data. It maintains *relationships* between objects. When you checkout a book, three objects are updated: the book (availability), the member (borrowed list), and the library (loan record). This is object-oriented thinking: objects collaborate to maintain system state.

Deeply Nested Structures: The Hospital System

```
const hospital = {
  departments: {
    emergency: {
      beds: [
        { id: "E1", patient: null, equipment: ["monitor", "oxygen"] },
        { id: "E2", patient: {
          id: "P101",
          name: "John Doe",
          vitals: { bp: "120/80", hr: 72, temp: 98.6 }
        },
        equipment: ["monitor"]
      ]
    },
    staff: [
      { id: "N001", name: "Nurse Joy", role: "RN", assignedBeds: ["E2"] },
      { id: "D001", name: "Dr. Smith", role: "MD", specialty: "Trauma" }
    ]
  }
},

pharmacy: {
  inventory: [
    { drug: "Morphine", stock: 50, unit: "mg", location: "Cabinet A" },
    { drug: "Epinephrine", stock: 10, unit: "vials", location: "Crash Cart" }
  ],
  prescriptions: [
    {
      patientId: "P101",
      medications: [
        { drug: "Morphine", dosage: "10mg", frequency: "q4h", status: "active" }
      ],
      prescribedBy: "D001",
      date: new Date()
    }
  ]
}
};

// Accessing: Navigate the tree
const patientName = hospital.departments.emergency.beds[1].patient.name;

// Safe navigation with optional chaining
const temp = hospital.departments?.emergency?.beds?.[1]?.patient?.vitals?.temp;

// Updating nested data (immutable)
```

```
function updateVital(hospital, bedId, vitalType, value) {
  return {
    ...hospital,
    departments: {
      ...hospital.departments,
      emergency: {
        ...hospital.departments.emergency,
        beds: hospital.departments.emergency.beds.map(bed => {
          if (bed.id !== bedId) return bed;
          return {
            ...bed,
            patient: {
              ...bed.patient,
              vitals: {
                ...bed.patient?.vitals,
                [vitalType]: value
              }
            }
          };
        })
      }
    }
  };
}
```

The challenge: Deep immutable updates are verbose. This is why libraries like Immer exist. But understanding the manual way builds your mental model.

Common Mistakes (That Even Experienced Developers Make)

Mistake 1: The Silent Mutation

```
function formatUser(user) {
  user.displayName = user.name.toUpperCase(); // Oops, mutated input
  return user;
}

const alice = { name: "Alice" };
const formatted = formatUser(alice);
// alice now has displayName – caller didn't expect this
```

Fix: Either return a new object, or clearly name the function `addDisplayNameToUser` to signal mutation.

Mistake 2: The Reference Leak

```
class TodoList {
  constructor() {
    this.todos = [];
```

```
}

getTodos() {
  return this.todos; // DANGER: Returns internal reference
}

const list = new TodoList();
const todos = list.getTodos();
todos.push({ text: "Hacked!" }); // Modifies internal state!
```

Fix: Return copies or frozen versions.

```
getTodos() {
  return [...this.todos]; // Shallow copy
  // Or: return Object.freeze([...this.todos]);
}
```

Mistake 3: The Nested Mutation

```
const state = {
  user: { profile: { name: "Alice" } }
};

const newState = { ...state };
newState.user.profile.name = "Bob"; // Mutates original!

// Fix: Deep copy (or use structuredClone)
const newState = structuredClone(state);
newState.user.profile.name = "Bob"; // Safe
```

Mistake 4: The Missing Property Assumption

```
function sendEmail(user) {
  // Assumes user.email exists
  const domain = user.email.split("@")[1]; // Crashes if email is undefined!
}

// Fix: Defensive coding
const domain = user.email?.split("@")?.[1] ?? "unknown";
```

Mistake 5: The Identity Confusion

```
const users = [{ id: 1, name: "Alice" }];
const alice = { id: 1, name: "Alice" };

users.includes(alice);           // false! Different object
users.find(u => u.id === 1);     // true – checks property, not identity
```

Real-World Example: E-Commerce Order Flow

```
// Entities
class Product {
  constructor(id, name, price, stock) {
    this.id = id;
    this.name = name;
    this.price = price;
    this.stock = stock;
  }

  isAvailable(quantity = 1) {
    return this.stock >= quantity;
  }

  reduceStock(quantity) {
    if (!this.isAvailable(quantity)) {
      throw new Error("Insufficient stock");
    }
    this.stock -= quantity;
  }
}

class CartItem {
  constructor(product, quantity) {
    this.product = product; // Reference to Product object
    this.quantity = quantity;
  }

  get subtotal() {
    return this.product.price * this.quantity;
  }
}

class ShoppingCart {
  constructor(customerId) {
    this.customerId = customerId;
    this.items = []; // Array of CartItem objects
    this.coupon = null;
    this.status = "active";
  }

  addProduct(product, quantity = 1) {
```

```
    if (!product.isAvailable(quantity)) {
      throw new Error("Out of stock");
    }

    const existing = this.items.find(item => item.product.id === product.id);

    if (existing) {
      if (!product.isAvailable(quantity)) {
        throw new Error("Not enough stock");
      }
      existing.quantity += quantity;
    } else {
      this.items.push(new CartItem(product, quantity));
    }
  }

  removeProduct(productId) {
    this.items = this.items.filter(item => item.product.id !== productId);
  }

  get total() {
    return this.items.reduce((sum, item) => sum + item.subtotal, 0);
  }

  get itemCount() {
    return this.items.reduce((sum, item) => sum + item.quantity, 0);
  }

  applyCoupon(coupon) {
    if (coupon.expires < new Date()) {
      throw new Error("Coupon expired");
    }
    this.coupon = coupon;
  }

  get discountedTotal() {
    if (!this.coupon) return this.total;
    return this.total * (1 - this.coupon.discount);
  }

  checkout() {
    // Validate
    if (this.items.length === 0) {
      throw new Error("Cart is empty");
    }

    // Reserve inventory
    this.items.forEach(item => {
      item.product.reduceStock(item.quantity);
    });

    // Create order
    const order = {
      id: generateId(),
```

```

        customerId: this.customerId,
        items: this.items.map(item => ({
            productId: item.product.id,
            name: item.product.name,
            price: item.product.price,
            quantity: item.quantity,
            subtotal: item.subtotal
        })),
        total: this.discountedTotal,
        coupon: this.coupon,
        status: "confirmed",
        createdAt: new Date()
    };

    this.status = "checked_out";
    return order;
}
}

// Usage
const laptop = new Product("p1", "Laptop", 999, 10);
const mouse = new Product("p2", "Mouse", 25, 50);

const cart = new ShoppingCart("cust_123");
cart.addProduct(laptop, 1);
cart.addProduct(mouse, 2);

console.log(cart.total);           // 1049
console.log(cart.itemCount);       // 3

const order = cart.checkout();
// laptop.stock is now 9
// mouse.stock is now 48

```

What this demonstrates:

- Objects referencing other objects (`CartItem` → `Product`)
- Arrays of objects (`items`)
- Methods that modify internal state (`addProduct`, `removeProduct`)
- Computed properties (`subtotal`, `total`, `discountedTotal`)
- State transitions (`active` → `checked_out`)
- Validation before mutation
- Creating new objects from existing ones (`checkout` creates an `Order` from `Cart`)

The Final Mental Model

When you write JavaScript, think in these terms:

1. **Entities** are objects. A user, a product, an order, a cart item.
2. **Collections** are arrays of entities. A cart has items. A library has books.

3. **Relationships** are references. A cart item references a product. An order references a customer.
4. **State changes** happen through methods that understand the entity's rules. You don't directly mutate `stock`. You call `reduceStock()` which checks availability first.
5. **Data flows** through functions that receive objects, transform them, and return new objects (or mutate when appropriate).
6. **Identity matters** for references. Two objects with the same data are not the same object.

Your goal: When you look at a problem, you should see entities, relationships, and state transitions—not just variables and functions.
